Data Cleaning

Bad data could be:

1. Wrong data
2. Data in wrong format
3. Duplicates
4. Empty cells/missing values
5. Outliers

In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:
```
df = pd.DataFrame({"Age":    [15,18,"18",19.4,"20+"],
                   "Gender": ["male","female","female","female","male"]
df
```

Out[2]:

	Age	Gender
0	15	male
1	18	female
2	18	female
3	19.4	female
4	20+	male

In [3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   Age     5 non-null      object
 1   Gender  5 non-null      object
dtypes: object(2)
memory usage: 212.0+ bytes
```

In [4]:
```
df["Age"].unique()
```

Out[4]:
```
array([15, 18, '18', 19.4, '20+'], dtype=object)
```

1. Wrong data

- Solution: Replace

In [5]:
```
df["Age"].replace({"20+":20},inplace=True)
df
```

Out[5]:

	Age	Gender
0	15	male
1	18	female
2	18	female
3	19.4	female
4	20	male

2.Wrong data type

- Solution: convert the datatype

In [6]:
```
df["Age"] = df["Age"].astype('float')
df
```

Out[6]:

	Age	Gender
0	15.0	male
1	18.0	female
2	18.0	female
3	19.4	female
4	20.0	male

3. Duplicates

- Solution: Remove

In [7]:
```
#to check the duplicated records
df.duplicated()
```

Out[7]:
```
0    False
1    False
2     True
3    False
4    False
dtype: bool
```

In [8]:
```
#total no. of duplicates in given data
df.duplicated().sum()
```

Out[8]: 1

In [9]:
```
#to extract duplicate records
df[df.duplicated()]
```

Out[9]:

	Age	Gender
2	18.0	female

In [10]:
```
#to extract non duplicated records

df[~df.duplicated()]
```

Out[10]:

	Age	Gender
0	15.0	male
1	18.0	female
3	19.4	female
4	20.0	male

In [11]:
```
# to remove the duplictes
df.drop_duplicates(inplace=True,ignore_index=True)
df
```

Out[11]:

	Age	Gender
0	15.0	male
1	18.0	female
2	19.4	female
3	20.0	male

Missing values

- Solution: Either remove or replace

In [12]:
```
df = pd.DataFrame({"Age":    [15,np.nan,24,19,20,22],
                   "Gender":["male",np.nan,"female", "female","male",np
df
```

Out[12]:

	Age	Gender
0	15.0	male
1	NaN	NaN
2	24.0	female
3	19.0	female
4	20.0	male
5	22.0	NaN

In [13]:
```
#to check the missing values records
df.isnull()
```

Out[13]:

	Age	Gender
0	False	False
1	True	True
2	False	False
3	False	False
4	False	False
5	False	True

In [14]:
```
# to check total missing values
df.isnull().sum()
```

Out[14]:
```
Age       1
Gender    2
dtype: int64
```

In [15]:
```
#to check percentage of missing values in each variable

df.isnull().sum()/len(df)*100
```

Out[15]:
```
Age       16.666667
Gender    33.333333
dtype: float64
```

Option 1. Remove the rows that contain missing values

In [16]:
```
df2 = df.dropna()
df2
```

Out[16]:

	Age	Gender
0	15.0	male
2	24.0	female
3	19.0	female
4	20.0	male

In [17]:
```
df1 = df.drop(columns=["Gender"])
df1
```

Out[17]:

	Age
0	15.0
1	NaN
2	24.0
3	19.0
4	20.0
5	22.0

Option 2: Replace the nan values

- fill with value
- Continous Variables ---> Replace with either Mean or Median
- Discrete Variables ---> Replace with Mode

In [18]:
```
# to fill with a value

df["Age"].fillna(20)
```

Out[18]:
```
0    15.0
1    20.0
2    24.0
3    19.0
4    20.0
5    22.0
Name: Age, dtype: float64
```

In [19]:
```
# to fill with mean
df['Age'].fillna(df["Age"].mean())
```

Out[19]:
```
0    15.0
1    20.0
2    24.0
3    19.0
4    20.0
5    22.0
Name: Age, dtype: float64
```

In [20]:
```
# to fill with median
df["Age"].fillna(df["Age"].median())
```

Out[20]:
```
0    15.0
1    20.0
2    24.0
3    19.0
4    20.0
5    22.0
Name: Age, dtype: float64
```

In [21]:
```
#to fill with mode
df["Gender"].fillna(df["Gender"].mode()[0])
```

Out[21]:
```
0      male
1    female
2    female
3    female
4      male
5    female
Name: Gender, dtype: object
```

Outliers

In [22]:
```
df= pd.DataFrame({"marks":[10,11,12,25,25,27,31,33,34,34,36,36,43,50,59
df
```

Out[22]:

	marks
0	10
1	11
2	12
3	25
4	25
5	27
6	31
7	33
8	34
9	34
10	36
11	36
12	43
13	50
14	59

Various ways of finding the outlier.

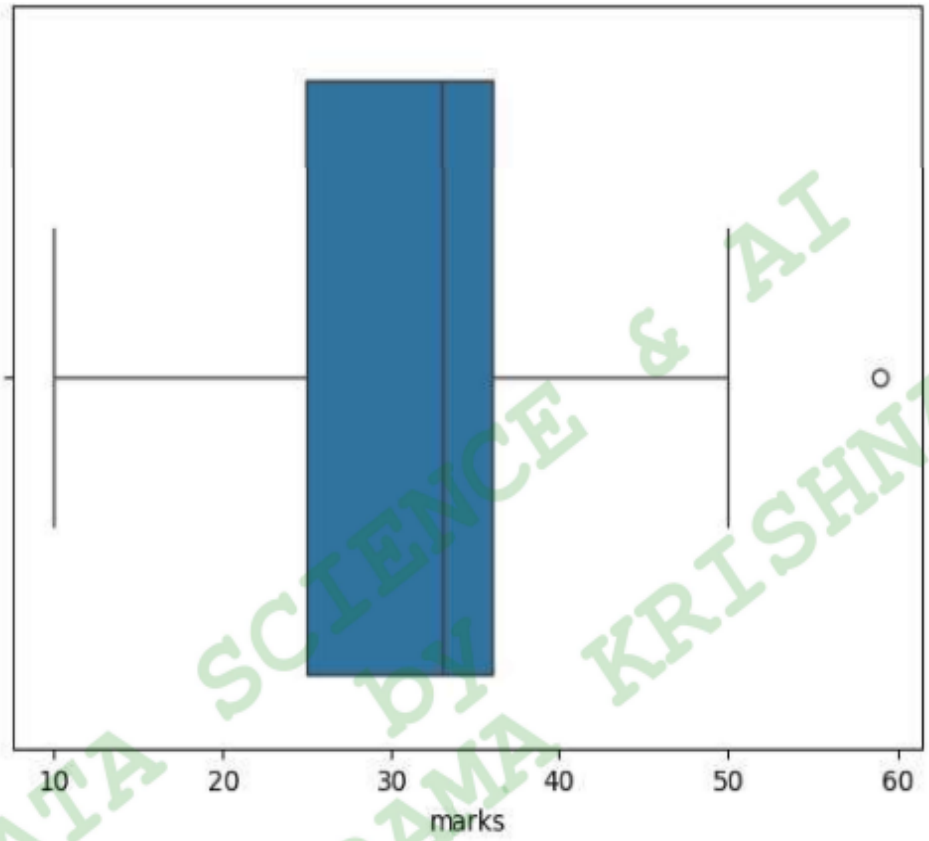
1. Boxplot

Identifying Outliers based on boxplot

In [23]:

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.boxplot(x=df["marks"])
plt.show()
```



Identifying Outliers based on IQR

In [24]:
```
#calculate Q1
Q1=df["marks"].quantile(0.25)
print("Q1:",Q1)

#calculate Q3
Q3=df["marks"].quantile(0.75)
print("Q3:",Q3)

#calculate IQR
IQR = Q3 - Q1
print("IQR:",IQR)

#Calculate lower limit of outlier
lower_limit = Q1 - (IQR * 1.5)
print("lower limit:",lower_limit)

#Calculate upper limit of outlier
upper_limit = Q3 + (IQR * 1.5)
print("upper limit:",upper_limit)
```

```
Q1: 25.0
Q3: 36.0
IQR: 11.0
lower limit: 8.5
upper limit: 52.5
```

Outliers Data

In [25]:
```
df[(df["marks"]<lower_limit) | (df["marks"]>upper_limit)]
```

Out[25]:

	marks
14	59

Solution: 3R Technique

1. Remove (remove the outliers from our dataset)
2. Replace the ouliers
   - Rectify or Replace --> (data entry error) ---> Ask and confirm it from the Data Engineering team.
   - Replace with upper limit & lower limit based on IQR
3. Retain (consider for analysis) ---> Treat them separately

Remove

In [26]:
```
df.drop(index=[14])
```

Out[26]:

	marks
0	10
1	11
2	12
3	25
4	25
5	27
6	31
7	33
8	34
9	34
10	36
11	36
12	43
13	50

Replace

- based on confirmation from data engineer team / based on research / based on domain expertise

> Replace based on statistics

- **winsorization** - replacing the outliers statiscally with lower_linit & upper_limit values

In [27]:
```
df['marks'] = df['marks'].clip(lower=8.5,upper=52.5)
df
```

Out[27]:

	marks
0	10.0
1	11.0
2	12.0
3	25.0
4	25.0
5	27.0
6	31.0
7	33.0
8	34.0
9	34.0
10	36.0
11	36.0
12	43.0
13	50.0
14	52.5

In [28]:
```
sns.boxplot(x=df["marks"])
plt.show()
```